

Teaching a Fracture Classifier Where to Look

CNN vs ScatNet, six XAI methods and a joint classifier–detector on FracAtlas

Silver Gjeka (VR527645) Sevastian Guzun (VR508193)

Department of Computer Science
Master's Degree in Artificial Intelligence
University of Verona

Visual Intelligence

Data: FracAtlas (Abdedeen et al., *Scientific Data*, 2023)
Code: github.com/silvergjeka22/bone-fracture-detection

Outline

Part I – Classify and explain

1. The problem and the data
2. CNN vs ScatNet (+ ResNet18)
3. Six XAI methods: faithful? on the fracture?

Part II – Teach it where to look

4. Three models: A, B and C, and their losses

Part III – Results

5. Do the explanations find the fracture?
6. Test X-rays
7. Every model side by side
8. Can our model help YOLO?
9. Limits and conclusion

Part I

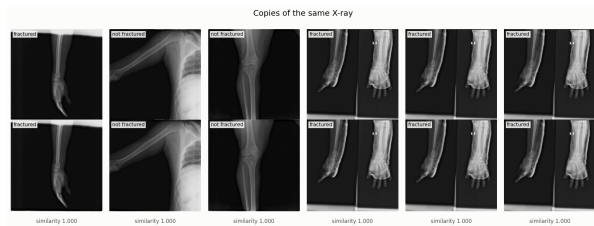
Classify and Explain

Which model sees fractures, and where does it look?

The Problem: Right, but for the Right Reason?

- A model can be **accurate for the wrong reasons**: a metal plate, a cast or the body part can predict “fractured” better than a thin fracture line.
- **XAI** draws a heatmap of the evidence. It helps only if it is **faithful** (shows what the model uses) **and** the model uses the **fracture**.
- FracAtlas has the **radiologists’ boxes**: we can **measure** where a model looks.
- Our question: can we **teach** a classifier to look at the fracture, without losing accuracy?

The Data: FracAtlas, Without Leakage

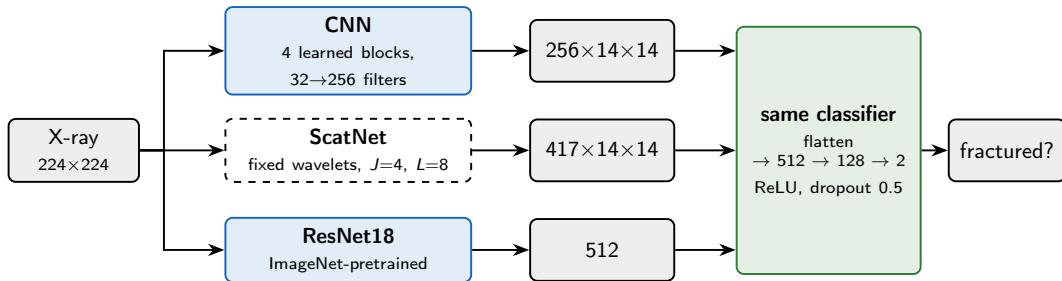


Copies of the same X-ray (flipped, resized, brighter) found automatically.

- 4,083 X-rays; **717 fractured** (1 in 6), 922 boxes.
- Grey, 224×224 (448 in our method), values in $[-1, 1]$.
- **8.6%** of the X-rays have a copy (158 groups): a split **keeps every group together**.
- Split $\approx 70/15/15$: **2,920** train, **577** val, **586** test (104 fractured).

Always saying “not fractured” already scores **82.3%**: look at **F1** and **fractures found**.

Three Feature Extractors, One Classifier



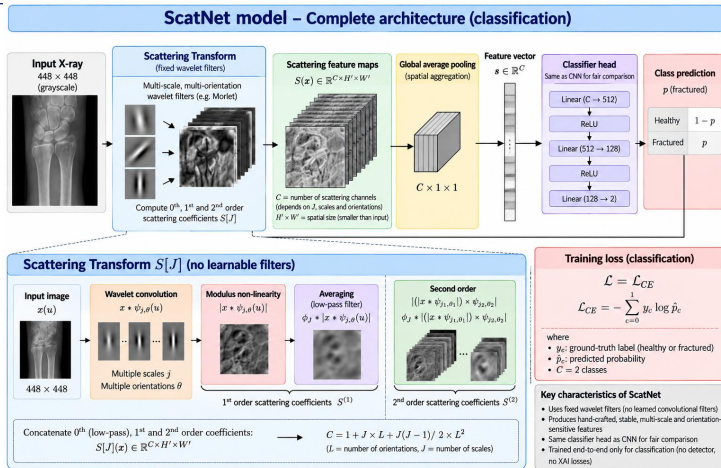
CNN: every filter **learned** from our 2,920 X-rays (first 7×7).

ScatNet: **nothing learned** before the classifier; stable to small deformations.

ResNet18: filters learned on **1.2M ImageNet** photos, then fine-tuned.

Only the classifier's **input size** differs (50,176 / 81,732 / 512). Weighted loss (1 in 6 fractured), 20 epochs, 5-fold grouped CV.

ScatNet: The Complete Architecture



In our run: input 224×224, $J=4$ scales, $L=8$ directions $\Rightarrow C = 1 + 32 + 384 = 417$ maps of 14×14, then log + BatchNorm, flattened into the same classifier (Bruna & Mallat, 2013; Kymatio).

CNN vs ScatNet (+ ResNet18): Results

	CNN	ScatNet	ResNet18
CV accuracy	73.0±9.0	77.5±2.9	89.0±1.0
CV F1	35.7±13.0	38.0±2.6	66.6±2.7
test accuracy	80.9	80.0	91.5
test F1	23.3	31.6	73.7
fractures found	16%	26%	67%
AUC	68.6	72.7	91.2
parameters	26.1M	41.9M	11.5M

McNemar: CNN $\approx$ ScatNet ($p = 0.55$); ResNet18 beats both ($p < 0.001$).

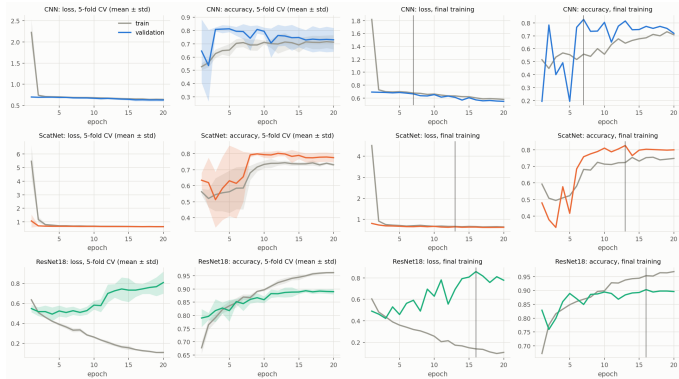
Why CNN and ScatNet fail

- Both **underfit** (loss stuck at 0.6) and stay **below** the 82.3% of “never fractured”.
- Only 511 fractured X-rays; a fracture is 1–2 px wide at 224 px.
- Flattening a 14×14 grid: 26–42M parameters in the classifier.

Why ResNet18 wins

- Pretrained edge and texture detectors that 2,920 X-rays cannot teach.
- It overfits after ≈ 10 epochs: the best val epoch is kept.

Learning Curves



Grey: train; colour: validation. Columns 1–2: 5-fold CV (mean $\pm$ std); columns 3–4: final training (vertical line = epoch kept).

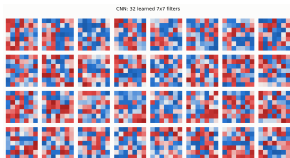
CNN (top) and ScatNet (middle)

- Train and val loss flatten at ≈ 0.6 – 0.7 after 2 epochs: **underfitting**, not overfitting.
- Accuracy jumps between epochs: the model flips between “all healthy” and guessing.

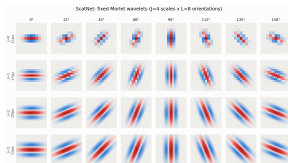
ResNet18 (bottom)

- Train loss keeps falling, val loss **rises after ≈ 10 epochs: overfitting.**
- Val accuracy still climbs slowly: we keep the **best val epoch**, not the last.

First-Layer Filters: What Each Model Uses



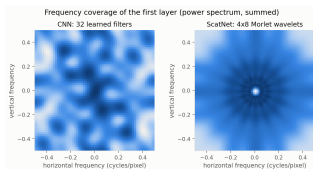
CNN: 32 filters 7×7 learned from scratch: **noisy**, no clear edges.



ScatNet: fixed Morlet wavelets, **every** scale and direction.



ResNet18: ImageNet filters: clean edges, bars and blobs.



- In frequency: the CNN covers the spectrum **unevenly** (2,920 X-rays are too few to learn clean filters); ScatNet covers it **evenly** by construction.
- But clean filters are not enough: ScatNet still fails, because it lacks a **learned** hierarchy above them.
- ResNet18 has both: clean filters **and** 17 learned layers above them.

Six XAI Methods: How Each One Works

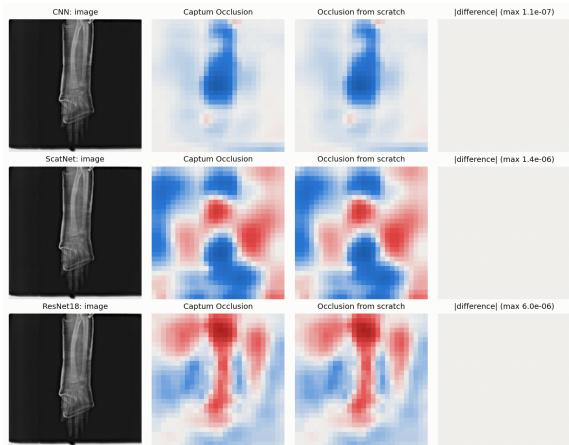
	how it works	strength / weakness	s/X-ray
Saliency	$ \partial z_c / \partial x $: score change for a tiny change of each pixel	fast / noisy, saturates	0.015
Integrated Gradients	gradients averaged on the path black $\rightarrow$ x, times $(x - x')$; sums to $z_c(x) - z_c(x')$	complete / slower	0.194
Guided Backprop	gradient, negative signals stopped at every ReLU	sharp / like an edge detector	0.010
Grad-CAM	last conv layer: channels weighted by their mean gradient, ReLU	class-specific / coarse; not on ScatNet	0.005
Occlusion	slide a black 16×16 window, record the score drop	honest / slow	0.767
LIME	switch random 16×16 patches off, fit a ridge regression	honest / random, slowest	1.377

Gradient methods ask “what if this pixel changed a little?”; **perturbation** methods (Occlusion, LIME) really **remove** parts of the X-ray (black = removed). Grad-CAM needs a learned convolutional layer, which ScatNet does not have.

Occlusion, Written From Scratch

Given: model f , X-ray x , class c , window $w=16$,
stride $s=8$
 $z \leftarrow f(x)_c$; $A \leftarrow 0$; $N \leftarrow 0$
for every window position (i, j) , in batches **do**
 $\hat{x} \leftarrow x$ with the window set to black
 $A[\text{window}] += z - f(\hat{x})_c$
 $N[\text{window}] += 1$
end for
return A/N ▷ mean drop per pixel

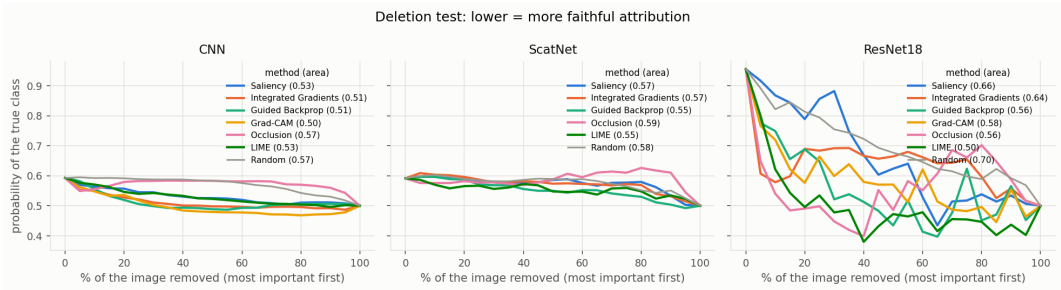
- Plain PyTorch, all positions scored in batches.
- Same answer as Captum on all three models: Pearson $r = 1.000$, largest difference $8.6 \cdot 10^{-6}$.



Our Occlusion, Captum's, and their difference.

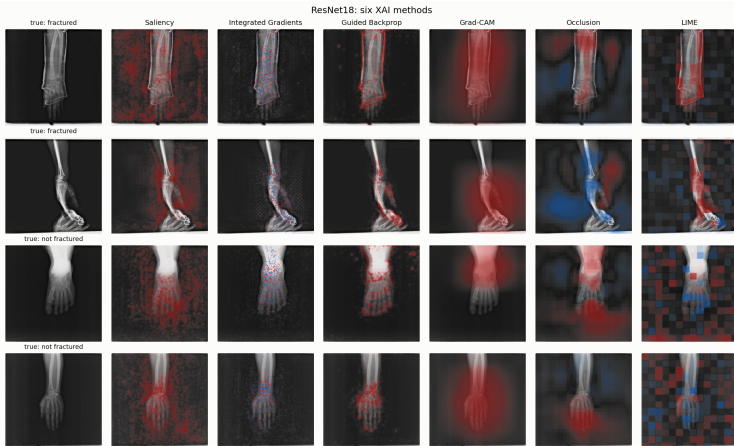
Is the Map Faithful? The Deletion Test

Remove the **most important** 16×16 patches **first** and follow the class probability: a faithful map makes it fall fast (**low area**). A random order is the reference.



The winner depends on the model: Grad-CAM for the CNN, LIME for ResNet18 (it removes patches, like the test); ScatNet is close to random for every method. Saliency and IG saturate in a deep net.

The Six Methods on Test X-rays (ResNet18)

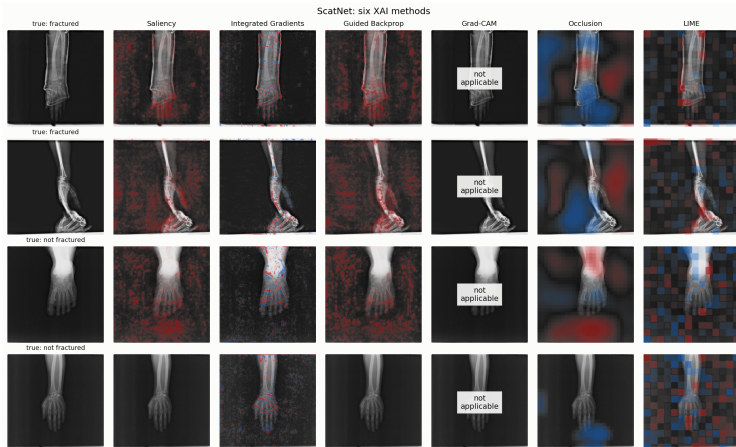


Two fractured and two healthy test X-rays; each method explains the model's answer. Red = evidence **for**, blue = **against**.

- **Saliency, IG:** scattered pixels, noisy.
- **Guided Backprop:** sharp, but traces every bone edge.
- **Grad-CAM:** one large blob (7×7 grid).
- **Occlusion, LIME:** patches, the most faithful in the deletion test.

The six maps disagree: there is no single “true” explanation.

The Six Methods on Test X-rays (ScatNet)

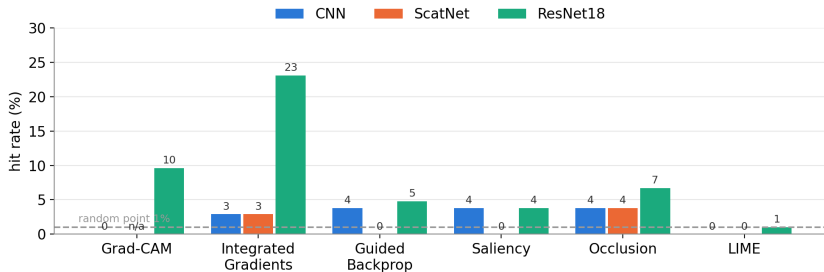


Two fractured and two healthy test X-rays; each method explains ScatNet's answer. Red = evidence for, blue = against.

- Grad-CAM: not applicable; ScatNet has no learned convolutional layer.
- Saliency, IG, Guided Backprop: diffuse responses along bone edges.
- Occlusion, LIME: broad regions and patches, without a clear fracture focus.

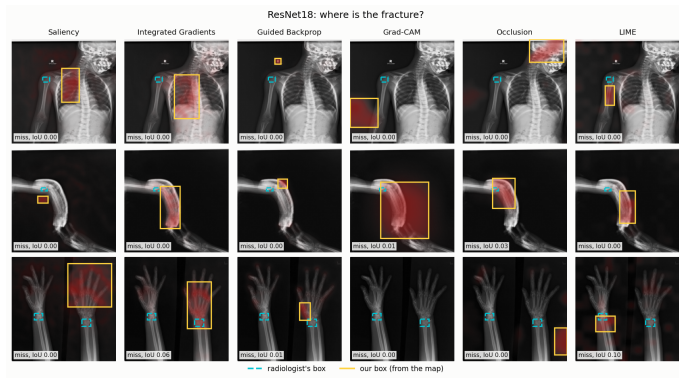
Across these examples, the maps do not consistently isolate the fracture.

Does the Map Point at the Fracture?



Rarely. 104 fractured test X-rays; a random point hits 1%. The best is Integrated Gradients on ResNet18: **23%**. CNN and ScatNet stay at 0–4% with every method.

Why So Rarely? ResNet18's Six Maps



Three fractured test X-rays. Dashed: the radiologist's box; yellow: the box we cut from each map.

- The boxes are **tiny** (1% of the image); fractures are 1–2 px at 224 px.
- A third of the fractures are **missed** by the model.
- The model uses **shortcuts**: the chest, the elbow, the hand; the faithful maps honestly show them.
- Grad-CAM on the 7×7 layer4 grid: one cell = 32 px, too coarse for a 1-px line.

⇒ Faithful maps are not enough: the **model** must be taught to look at the fracture.

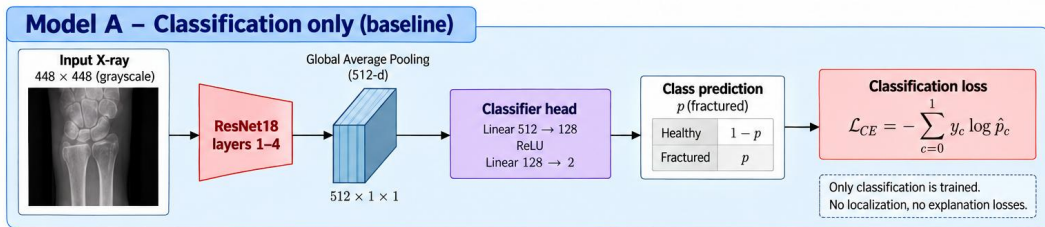
Part II

Teach the Model Where to Look

One network that classifies, detects, and is trained on its own explanation

Model A: Classifier Only (Baseline)

Three models, one network: the same ResNet18 at 448 px, same data, 20 epochs and seed. **Only the training loss changes** from A to B to C.



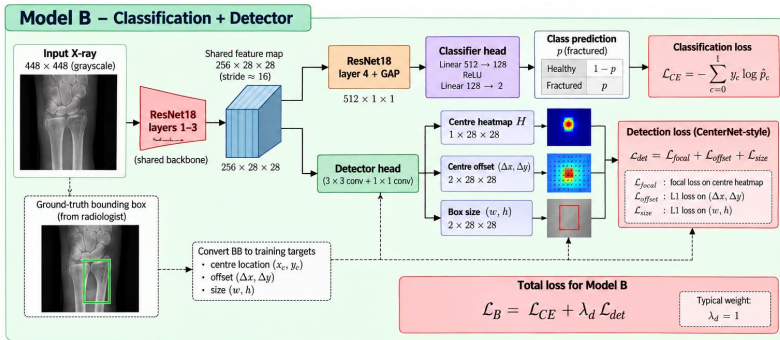
Loss: cross-entropy only, $\mathcal{L}_A = \mathcal{L}_{CE}$, batches of 16, half fractured.

- 448 px: $4\times$ the pixels, thin fractures survive.
- Boxes come from a **separate** YOLO.

Why it falls short: nothing in its loss says **where**.

- Grad-CAM on the fracture: **35%**.
- Blur the fracture: $p(\text{fractured})$ 81% $\rightarrow$ **84%**: it does not need the fracture.

Model B: Classifier + Detector



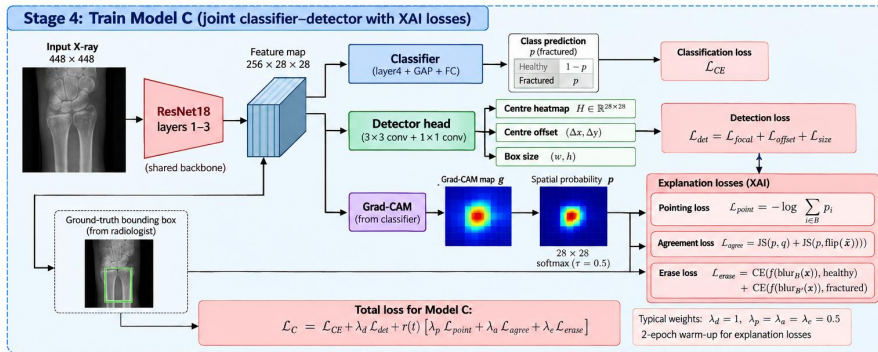
How: a small **detector head** (CenterNet, Zhou et al. 2019) on the **same** layer3 grid as the classifier.

Detection loss: focal loss on the centre heatmap + L1 on offset and size, from the radiologists' boxes.

Why it helps: the shared grid must learn **where** fractures are: Grad-CAM 35% $\rightarrow$ **56%**.

Why not enough: the classifier is still free to decide from elsewhere; nothing ties *its* explanation to the box.

Model C: Our Joint Model (B + Explanation Losses)



Point: Grad-CAM (on the same 28×28 grid), softmax → p ; maximise the mass inside the box. A smooth hit rate; the softmax never loses its gradient.

Agree: Grad-CAM ↔ detector map, X-ray ↔ mirrored X-ray (Jensen–Shannon). SimCLR-like: two views, one place; it updates **both** heads.

Erase: blur the fracture ⇒ “healthy”; blur a random patch ⇒ still “fractured”. Moves the **decision**, not only the map.

Grad-CAM, not IG, in the loss: one backward pass per step. Explanation losses start after a 2-epoch warm-up.

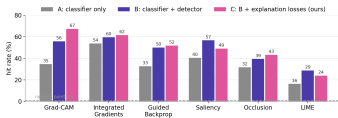
Part III

Results

A vs B vs C, test X-rays, and YOLO

A vs B vs C: Where They Look, and Why

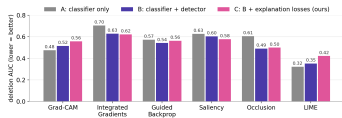
1. On the fracture?



Hit rate, 104 fractured test X-rays.

- **B** lifts every method (mean 35 $\rightarrow$ 48%).
- **C**: Grad-CAM 35 $\rightarrow$ 67% ($p < 0.001$), the best method.
- Only Grad-CAM is in C's loss, yet the others improve too.

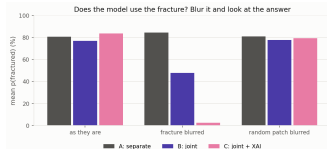
2. Still faithful?



Deletion AUC, lower = more faithful.

- Mean of six: 0.55 (A), 0.52 (B), 0.54 (C); random ≈ 0.70 .
- **Faithfulness is kept**; LIME stays the most faithful.

3. Does the decision follow?

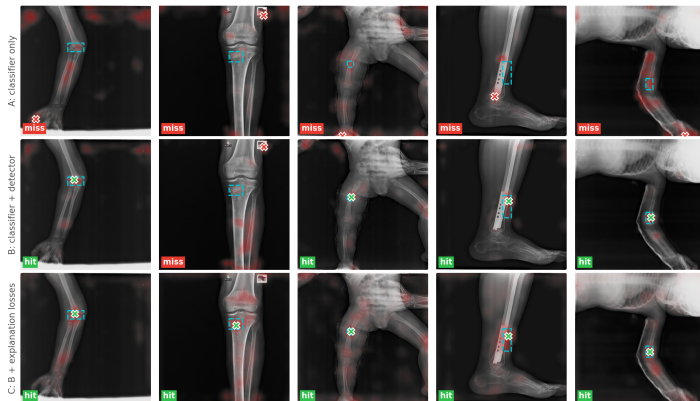


$p(\text{fractured})$: as is / fracture blurred / random patch blurred.

- **A**: 81 / 84 / 81: it ignores the fracture.
- **C**: 84 / 2.5 / 79: it decides from the fracture.

The better maps are **not cosmetic**: C looks at the fracture, stays faithful, and its answer changes when the fracture is removed.

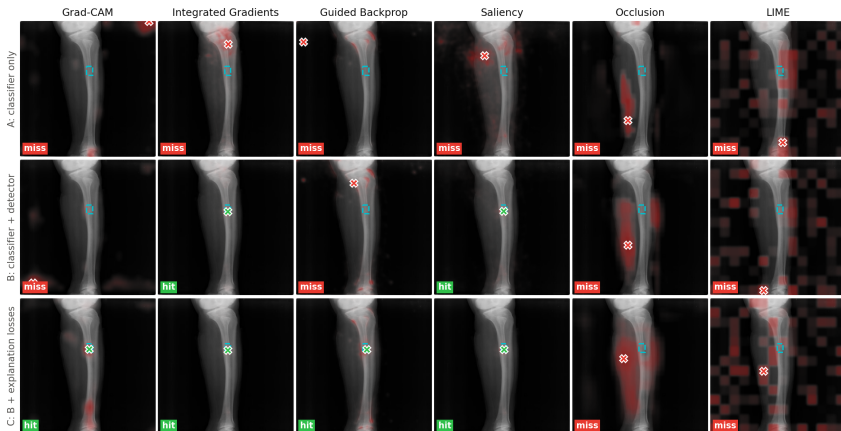
Test



Grad-CAM of A, B, C on five **new** fractured test X-rays.

- Red: where the model looks; **X**: its hottest point; dashed: the radiologist's box.
- **A** looks at a label, the hand, the foot, an image corner.
- **B** already finds 4 of 5; **C** finds all 5.
- **C** is one of **35** such X-rays; the opposite (**A** hits, **C** misses) happens **once**.

Test: One X-ray, Six Methods, Three Models



A thin lower-leg fracture. **A:** all six methods miss. **B:** IG and Saliency find it. **C:** four of six, including gradient methods outside its loss. Occlusion and LIME (32-px patches) stay too coarse for a 1-cell box.

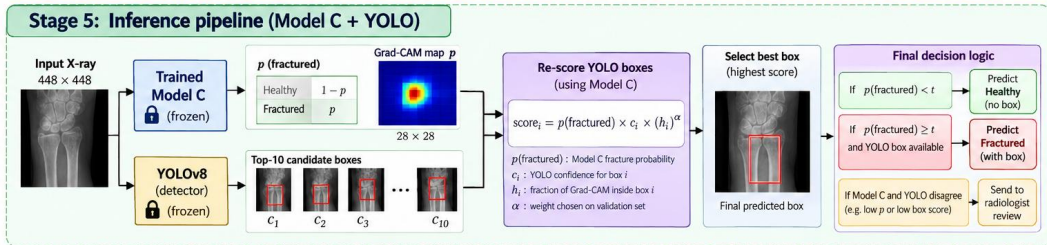
Every Model Side by Side

	what it learns	acc.	F1	best hit	blur drop	good at
CNN	filters from 2,920 X-rays	80.9	23.3	4%	–	– (underfits)
ScatNet	only the classifier	80.0	31.6	4%	–	stable features, no fractures
ResNet18	fine-tunes ImageNet	91.5	73.7	23%	–	classification
A: classifier only	same, 4× the pixels	93.3	81.0	54%	–4	finding fractures
B: + detector	+ where the centre is	94.2	82.3	60%	29	fewest false alarms
C: ours	+ where it looks, blur test	93.0	80.4	67%	81	looking at the fracture
YOLOv8	only boxes	–	–	79% (box)	–	placing the box

“best hit”: best of the six maps (for YOLO: its most confident box). “blur drop”: points of $p(\text{fractured})$ lost when the fracture is blurred. Same 586 test X-rays for all.

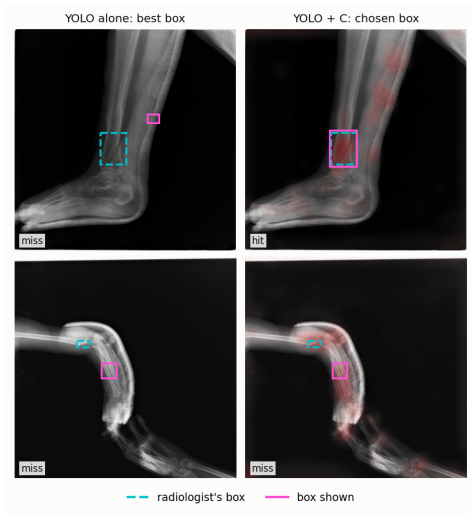
A, B and C are within each other's confidence intervals (± 2 points; a second run gave 91.1 / 93.3 / 93.9%). Our method does not change **how often** the model is right. It changes **why**.

YOLO + C: YOLO Proposes, C Decides



- **YOLOv8s** (COCO-pretrained, fine-tuned on the boxes): its best box is on the fracture for **79%** of the X-rays, better than any explanation (67%).
- Each of YOLO's 10 boxes is re-scored with C:
 $p \cdot c^{1-\alpha} \cdot h^\alpha$, $\alpha = 0.5$ chosen on val.
- C and YOLO **agree** $\Rightarrow$ decide (76% of the X-rays, **96.6%** right); they disagree $\Rightarrow$ radiologist review (24%).
- Fractures missed: **5.8%** (6 of 104).

Test: YOLO vs YOLO + C



Top: YOLO's best box is off the fracture; C's Grad-CAM lifts the right one: miss → **hit**.

Bottom: both miss: C can only choose among YOLO's boxes.

Recap on all test X-rays

- **Placing the box:** no gain: 76% vs 79% (not significant). YOLO's own ranking is already good.
- **Fewer false boxes:** yes: AP@0.5 on all X-rays **40.7** → **46.9**, boxes on healthy X-rays fall to the bottom.
- **Pipeline:** when C and YOLO agree it decides 76% of the X-rays, **96.6%** right; the rest goes to a radiologist.

⇒ For *where*, use YOLO; for *whether*, use C.

Limitations and Lessons

Limitations

- One dataset; one seed per run (two full runs of A, B, C).
- 104 fractured test X-rays: 4 points = 4 X-rays. Deletion test on 10 X-rays.
- Loss weights, thresholds, α set once, not tuned.
- The pointing game is strict: one point, a tiny box.
- C is trained with the blur it is tested with (controls: random patch, five untrained maps).
- Casts and metal plates still fool every model.
- The built-in detector stays far behind YOLO (AP 22–31% vs 53%).

Lessons

- **Accuracy is not enough:** 80% can mean “finds a fifth of the fractures”.
- **Pretraining and resolution** beat hand-crafted invariance.
- A faithful map can still show a **shortcut**; the model must be **taught** to look.
- **Score the layer you train**, and test the decision, not only the map.
- A **specialist** stays best at its job: use YOLO for boxes, our model as a second opinion.

Conclusion

- **CNN vs ScatNet**: statistically tied and close to “never fractured”; a pretrained ResNet18 reaches **91.5%**.
- **Six XAI methods**: the most faithful depends on the model (Grad-CAM for the CNN, LIME for ResNet18); our Occlusion equals Captum; but the maps point at the fracture at most **23%** of the time.
- **Our joint model C** (classifier + detector + explanation losses, 448 px):
 - keeps the accuracy (**93.0%**);
 - Grad-CAM hits the fracture **67%** of the time instead of 35%, and the five other methods improve too;
 - blurring the fracture now changes the answer (84% → **3%**).
- **YOLO** remains the best box finder (79%); our model helps it by removing false boxes (AP on all X-rays 40.7 → 46.9), and together they decide 76% of the X-rays with **97%** accuracy.

Thank you for your attention!

Code, results and figures: github.com/silvergjecha22/bone-fracture-detection